

Manipuler des fichiers

Astuces

- Des noms de fichiers doivent être courts. Évitez les caractères accentués, les espaces et les signes typographiques spéciaux.
- Les fichiers en question seront créés et/ou recherches par Python dans le répertoire courant.
- Pour forcer Python à changer son répertoire courant :

```
>>> from os import chdir  
>>> chdir("/home/jules/exercices")
```

```
>>>import os
```

```
>>>os.chdir("C:/python")
```

Le module **os** contient toute une série de fonctions permettant de dialoguer avec le système d'exploitation (**os** = *operating system*), quel que soit celui-ci.

La commande **chdir** provoque le changement de repertoire (**chdir** = *change directory*).

Module os

- Le module os : pour interagir directement avec le système d'exploitation

- `os.path.join()` pour recréer aisément des chemins

```
import os
```

```
rep = "./fichiers_tests" #le répertoire avec les fichiers à lire
```

```
liste_fichiers = os.listdir(rep) # liste des fichiers contenus dans le
```

répertoire

```
print(liste_fichiers)
```

Importer un module

Deux possibilités:

1.

```
>>> import os
>>> rep_cour = os.getcwd()
>>> print rep_cour
C:\Python22\essais
```

getcwd() est la fonction du module **os** qui renvoie le nom du répertoire courant (*getcwd = get current working directory*).

2.

```
>>> from os import getcwd
>>> rep_cour = getcwd()

>>> print(rep_cour)
C:\Python31\essais
```

Nous n'avons importé du module **os** que la fonction **getcwd()**. Importée de cette manière, la fonction s'intègre à notre propre code comme si nous l'avions écrite nous-mêmes.

Importer un module

- importer plusieurs fonctions du meme module :

```
>>> from math import sqrt, pi, sin, cos
```

- importer toutes les fonctions d'un module :

```
>>> from math import *
```

Écriture séquentielle dans un fichier

- L'accès aux fichiers est assuré par l'intermédiaire d'un objet-interface particulier: ***objet-fichier***
- On crée cet objet à l'aide de la fonction intégrée **open()** qui renvoie un objet doté de méthodes spécifiques permettant de lire et écrire dans le fichier.

La fonction **open()**

- attend deux arguments, qui doivent être des chaînes de caractères;
- le premier argument est le nom du fichier à ouvrir, et le second est le mode d'ouverture.
 - 'a' indique qu'il faut ouvrir ce fichier en mode « ajout » (*append*) : les données à enregistrer doivent être ajoutées à la fin du fichier
 - 'w' (*write*) : Python crée toujours un nouveau fichier (vide), et l'écriture des données commence à partir du début de ce nouveau fichier. S'il existe déjà un fichier de même nom, celui-ci est effacé

La méthode **write()**

- réalise l'écriture proprement dite.
- les données à écrire doivent être fournies en argument. Ces données sont enregistrées dans le fichier les unes à la suite des autres.
- chaque nouvel appel de **write()** continue l'écriture à la suite de ce qui est déjà enregistré.

La méthode **close()**

- ferme le fichier qui est désormais disponible pour tout usage.

Écriture séquentielle dans un fichier

```
>>> obFichier = open('Monfichier', 'a')
>>> obFichier.write('Bonjour, fichier !')
>>> obFichier.write("Quel beau temps, aujourd'hui !")
>>> obFichier.close()
>>>
```

- si le fichier n'existe pas encore, il sera créé automatiquement
- si le fichier existe et contient déjà des données, les caractères qu'on y enregistre viendront s'ajouter à la suite de ceux qui s'y trouvent

Lecture séquentielle d'un fichier

```
>>> ofi = open('Monfichier', 'r')
>>> t = ofi.read()
>>> print(t)
Bonjour, fichier !Quel beau temps, aujourd'hui !
>>> ofi.close()
```

Méthode **read()**

- lit les données présentes dans le fichier et les transfère dans une variable de type chaîne de caractères.
- si on utilise cette méthode sans argument, la totalité du fichier est transférée.
- si le fichier n'existe pas, on obtient un message d'erreur.

```
>>> ofi = open('Monfichier','r')
```

```
IOError: [Errno 2] No such file or directory: 'Monfichier'
```


Lecture séquentielle d'un fichier

Méthode **read()**

- peut être utilisée avec un argument qui indiquera combien de caractères doivent être lus, à partir de la position déjà atteinte dans le fichier :

```
>>> ofi = open('Monfichier', 'r')
```

```
>>> t = ofi.read(7)
```

```
>>> print(t)
```

```
Bonjour
```

```
>>> t = ofi.read(15)
```

```
>>> print(t)
```

```
, fichier !Quel
```

- s'il ne reste pas assez de caractères au fichier pour satisfaire la demande, la lecture s'arrête à la fin du fichier :

```
>>> t = ofi.read(1000)
```

```
>>> print(t)
```

```
beau temps, aujourd'hui !
```

- si la fin du fichier est déjà atteinte, **read()** renvoie une chaîne vide :

```
>>> t = ofi.read()
```

```
>>> print(t)
```

- n'oubliez pas de refermer le fichier après usage :

```
>>> ofi.close()
```

Fichier texte : créer un fichier texte de quatre lignes

- le marqueur de fin de ligne `\n` marque les endroits où l'on souhaite séparer les lignes de texte dans l'enregistrement. Sans ce marqueur, les caractères seraient enregistrés les uns à la suite des autres

```
>>> f = open("Fichier texte", "w")
>>> f.write("Ceci est la ligne un\nVoici la ligne deux\n")
>>> f.write("Voici la ligne trois\nVoici la ligne quatre\n")
>>> f.close()
```

- La méthode **`readline()`**, ne lit qu'une seule ligne à la fois (en incluant le caractère de fin de ligne)

```
>>> f = open('Fichier texte', 'r')
>>> t = f.readline()
>>> print(t)
Ceci est la ligne un
>>> print(f.readline())
Voici la ligne deux
```

- La méthode **`readlines()`** transfère toutes les lignes restantes dans une liste de chaînes :

```
>>> t = f.readlines()
>>> print(t)
['Voici la ligne trois\n', 'Voici la ligne quatre\n']
>>> f.close()
```

La méthode **readlines()**

- permet de lire l'intégralité d'un fichier en une instruction seulement.
- cela n'est possible que si le fichier à lire n'est pas trop gros : puisqu'il est copié intégralement dans une variable, c'est-à-dire dans la mémoire vive de l'ordinateur.
- si l'on traite de gros fichiers, utiliser plutôt la méthode **readline()** dans une boucle

Fichier texte

readline()

- est une méthode qui renvoie une *chaîne de caractères*
- à la fin du fichier, elle renvoie une chaîne vide

readlines()

- renvoie une *liste*.
- à la fin du fichier, il renvoie une liste vide.

Lecture par morceau

```
def copieFichier(source, destination):  
    "copie intégrale d'un fichier"  
    fs = open(source, 'r')  
    fd = open(destination, 'w')  
    while 1:  
        txt = fs.read(50)  
        if txt == "":  
            break  
        fd.write(txt)  
    fs.close()  
    fd.close()  
    return
```

Fonction **copieFichier** lira le fichier morceau par morceau (50 caractères par 50), jusqu'à ce que l'on ait atteint sa fin. On utilisera l'instruction **break** lorsque la fin du fichier aura été atteinte.

```
copieFichier('Monfichier', 'Tonfichier')
```

- Une boucle **while** devrait boucler infiniment, puisque la condition de continuation reste toujours vraie.
=> On fait appel à l'instruction **break**

Enregistrement et restitution de variables diverses

- L'argument de la méthode **write()** utilisée avec un fichier texte doit être une chaîne de car.
- On ne peut pas enregistrer d'autres types de valeurs qu'en les transformant d'abord en chaînes de car. à l'aide de la fonction intégrée **str()**

```
>>> x = 52  
>>> f.write(str(x))
```

Vérifier l'existence du fichier

- On demande à l'utilisateur d'entrer un nom de fichier.
- Si le fichier n'existe pas, on veut qu'un avertissement soit affiché, et que l'utilisateur puisse essayer d'entrer un autre nom.

```
filename = input("Veuillez entrer un nom de fichier : ")
try:
    f = open(filename, "r")
except:
    print("Le fichier", filename, "est introuvable")
```

On peut inclure ce test aussi dans une fonction:

```
def existe(fname):
    try:
        f = open(fname, 'r')
        f.close()
        return 1
    except:
        return 0

filename = input("Veuillez entrer le nom du fichier : ")
if existe(filename):
    print("Ce fichier existe bel et bien.")
else:
    print("Le fichier", filename, "est introuvable.")
```

with

- On peut faire des erreurs quand on lit un fichier, on écrit dans le fichier ...
- With permet de créer un « context manager » (gestionnaire de contenu) qui vérifie que le fichier est ouvert et fermé; même si des erreurs se produisent pendant le bloc
- Il est inutile de fermer le fichier à la fin du bloc with. Python va le faire tout seul, qu'une exception soit levée ou pas

with open (mon_fichier, mode_ouverture) as variable :

⇒ with : prélude un bloc dans lequel on manipule le fichier

⇒ as : en tant que

⇒ variable : contiendra notre objet, si la variable n'existe pas, python la crée

with open ("fichier.txt", "r") as mon_fichier :

texte=mon_fichier.read()

⇒ Si une exception se produit, le fichier sera tout de même fermé à la fin du bloc

Bonus pour les avancés

Obtenir le contenu d'un fichier

- `readline()` : retourne 1 ligne du fichier(`str()`)
- `readlines()` : retourne une liste avec toutes les lignes du fichier (`list()`)

Récupération de toutes les lignes

```
with open(os.path.join(rep,liste_fichiers[0]),"r",encoding="utf-8") as file_opened:  
    lignes = file_opened.readlines()  
print(lignes[:2])
```

Récupération de quelques lignes

```
with open(os.path.join(rep,liste_fichiers[0]),"r",encoding="utf-8") as file_opened:  
    ligne1 = file_opened.readline()  
    ligne2 = file_opened.readline()  
print(ligne1,ligne2)
```

Lecture ligne à ligne

```
print("*****")  
with open(os.path.join(rep,liste_fichiers[2]),"r",encoding="utf-8") as file_opened:  
    for line in file_opened:  
        print(line)
```

with open()

with open() permet aussi d'écrire des fichiers !

- write() ne prend en argument que des chaînes de caractères
- on peut utiliser .join() pour concaténer tous les éléments d'une liste entre eux

```
rep_out = "./sorties"
```

```
liste = ["pomme","banane"]
```

```
try: # on essaye de créer un répertoire de sortie
```

```
    os.mkdir(rep_out)
```

```
except OSError : #si le répertoire de sortie spécifié existe déjà
```

```
    pass
```

```
file_out = "fichier_de_sortie.txt"
```

```
file_out2 = 'fichier2.txt'
```

```
with open(os.path.join(rep_out,file_out),"w",encoding="utf-8") as out:
```

```
    out.write("Bonjour bonjour !")
```

```
with open(os.path.join(rep_out,file_out2),"w",encoding="utf-8") as out:
```

```
    out.write(" ".join(liste))
```

```
with open(os.path.join(rep_out,file_out),"a",encoding="utf-8") as out:
```

```
    out.write("$$$".join(liste))
```

```

import os
rep = "./fichiers_tests"    #le répertoire avec les fichiers à lire, créer le dossier fichierstests à la main
liste_fichiers = os.listdir(rep)    # liste des fichiers contenus dans le répertoire
print(liste_fichiers)

# for file in liste_fichiers: # on pourrait réutiliser la liste des fichiers créée ci-dessus
for file in os.listdir(rep): # pour chaque fichier
    print("Nom du fichier :", file, type(file))
    file_path = os.path.join(rep,file) # reconstitution du chemin du fichier
    #print("Chemin du fichier :", file_path, type(file_path))
    #-----
    file_opened = open(file_path, "r", encoding="utf-8") # ouverture en lecture
    #print("Objet fichier ouvert :", file_opened, type(file_opened))
    file_read = file_opened.read() # récupération du contenu
    #print("Contenu du fichier :",type(file_read),"\\n*****\\n", file_read, )
    file_opened.close() # fermeture du fichier
    #print("\\n*****\\n")
    #-----
    with open(file_path, "r", encoding="utf-8") as file_opened: # ouverture
        file_read2 = file_opened.read() # récupération du contenu
        #print("Contenu du fichier :",type(file_read2),"\\n$$$$$$\\n", file_read2)
        #print("\\n$$$$$$\\n")
try : # Pour vérifier qu'on obtient bien le même résultat avec les deux méthodes
    assert file_read == file_read2
    #assert file_read == "patate" #certainement faux
except AssertionError :
    print("/!\\ Les contenus sont différents ! /!\\")

```

A faire

- Recopier un fichier texte dans un autre fichier, en omettant toutes les lignes qui commencent par un caractère #
- Comparer les contenus de deux fichiers et signaler la première différence rencontrée.
- A partir de deux fichiers préexistants, construire le troisième qui contienne alternativement un élément de 1er, un élément du 2^{ème}, un élément de 1er, un élément du 2^{ème}... et ainsi de suite jusqu'à atteindre la fin de l'un des deux fichiers originaux. Compléter le troisième fichier avec les éléments restant sur l'autre.
- Reprendre l'ex. où il fallait écrire un script qui crée et consulte le dictionnaire avec les informations sur les noms, l'âge et la taille des gens. Complétez cet ex. en lui ajoutant deux fonctions : l'une pour enregistrer le dictionnaire résultant dans un fichier texte, et l'autre pour reconstituer ce dictionnaire à partir du fichier correspondant.

Travail à faire

- Créer et de relire un fichier texte. Le programme demandera d'abord à l'utilisateur d'entrer le nom du fichier. Ensuite il lui proposera le choix, soit d'enregistrer de nouvelles lignes de texte, soit d'afficher le contenu du fichier.

L'utilisateur devra pouvoir entrer ses lignes de texte successives en utilisant simplement la touche <Enter> pour les séparer les unes des autres.

Pour terminer les entrées, il lui suffira d'entrer une ligne vide (utiliser la touche <Enter> seule).

L'affichage du contenu devra montrer les lignes du fichier séparées les unes des autres de la manière la plus naturelle (les codes de fin de ligne ne doivent pas apparaître).